Active Rules for Embedded Databases: Lightweight Event-Driven Query Processing for Resource-Constrained Devices

MacKenzie Richards and Ramon Lawrence[a]
University of British Columbia - Okanagan Campus, Kelowna, BC, Canada

Keywords: embedded database, active database, event-condition-action rules, IoT, edge computing, time series database

Abstract: Resource-constrained Internet of Things (IoT) and edge devices increasingly require autonomous, real-time decision making without relying on cloud services or energy-intensive polling loops. This paper presents a lightweight active rule mechanism for event-driven query processing in an embedded database environment. The design adds event-condition-action style rules that are evaluated on insertion, supports sliding-window aggregates and user-defined callbacks, and reuses the existing query framework to provide active database functionality with low overhead. Existing active database and stream processing systems support event-condition-action (ECA) rules, but they target server or embedded Linux environments and are not designed for microcontroller-class devices with only a few kilobytes of memory. Experimental results show that active rules are practical on embedded hardware. On an Arduino Due, a representative frost-detection rule using a 10-record sliding window sustained about 1909 inserts per second with an average overhead of 185 µs per insert, and performance scaled predictably with window size. Comparison with manually issuing equivalent user-level queries after each insert showed nearly identical performance, indicating that the measured cost is dominated by query execution rather than the active rule interface itself. On a computer, the active rule implementation processed a 10-record window query in 0.00139 ms per insert, compared to 1.897 ms per insert for InfluxDB with Kapacitor, highlighting the advantage of local embedded rule execution over a client-server stream processing architecture. These results demonstrate that active database functionality can be implemented efficiently on highly constrained devices and can support low-latency local monitoring, alerting, and actuation for edge and IoT workloads.

1 INTRODUCTION

Embedded devices increasingly collect and process time series data in monitoring, control, and edge analytics applications. Prior work has shown that lightweight database support can enable efficient storage and query processing even on highly resource-constrained hardware. However, many of these applications require more than passive data storage and retrieval. They require the system to react immediately when incoming data satisfies application-defined conditions. For instance, in vineyards, temperature and humidity sensors can monitor changes in environmental conditions, such as the rate of temperature drop, to autonomously detect and mitigate frost pockets by triggering preventive actions to heat the area.

A conventional approach is to repeatedly poll the database or issue user-level queries after each insert to determine whether a condition has been met. This increases CPU overhead, complicates application logic, and delays response time because detection depends on when the next query is issued. For embedded systems, these costs are especially important because they directly affect latency, energy use, communication requirements, and software complexity. In contrast, active databases support event-condition-action (ECA) rules that automatically evaluate conditions and execute actions when relevant events occur.

This paper presents a lightweight active rule mechanism for an embedded database environment. The main contribution is the addition of ECA-style active rules that are evaluated on insertion and support sliding-window aggregates, threshold predicates, and callback actions. The design reuses the existing query framework while adding practical active database functionality suitable for embedded systems. This allows reactive behavior to be registered with the system rather than repeatedly implemented through user-level queries in application code.

[a] https://orcid.org/0000-0002-6779-4461

The contribution is not simply adding trigger-like functionality to a general-purpose database. Instead, the goal is to show that active query processing can be supported efficiently on resource-constrained devices that are not supported by current research and commercial systems and can execute directly where data is generated. This is important because edge and IoT deployments often rely on client-server architectures in which measurements are transmitted over the network to a remote database or stream processor before an action is taken. This introduces communication and coordination overhead that can be avoided when rule evaluation is performed locally on the device.

The evaluation considers two questions. First, can active rules be executed efficiently on embedded hardware? To answer this, the paper evaluates a representative frost-detection rule on an Arduino Due and studies the effect of sliding-window size. Second, how does local active processing compare with a client-server active time series system? To answer this, the paper compares the proposed approach with InfluxDB and Kapacitor on a PC platform. The results show that active rules are practical on embedded hardware, that their cost is dominated by the underlying query execution rather than the rule interface itself, and that local rule execution can avoid the overheads of server-based stream processing.

The contributions of this paper are:

1. the design of a lightweight active rule model for event-driven query processing on resource-constrained devices,
2. an implementation that integrates active rule execution into the insert path and supports sliding-window aggregates and callback actions, and
3. an experimental evaluation showing that active rules can execute efficiently on embedded hardware and can provide a low-latency alternative to client-server active processing.

The remainder of the paper is organized as follows. Section 2 reviews active databases and related systems. Section 3 presents the active rule design and implementation. Section 4 evaluates performance on embedded hardware and compares the approach with InfluxDB and Kapacitor. Section 5 discusses limitations and future work. Section 6 concludes.

2 Background and Related Work

2.1 Active Databases and ECA Rules

Traditional databases follow a pull-based model in which applications explicitly issue queries to retrieve or update information. This model works well for general-purpose data management, but reactive behavior must usually be implemented outside the database by repeatedly polling for changes or issuing user-level queries after updates. In embedded and edge settings, this can increase latency, waste computation, and complicate application logic.

Active databases extend this model by allowing the system to react automatically when specified events occur. The standard abstraction is an event-condition-action (ECA) rule (Dittrich et al., 1995): an *event* triggers the rule, a *condition* is evaluated, and an *action* is executed if the condition holds. In conventional database systems, ECA rules are often implemented using triggers associated with operations such as `INSERT`, `UPDATE`, and `DELETE`. More expressive active database systems may also support temporal events, composite events, persistent rule bases, and conflict resolution among multiple triggered rules.

For this work, the most relevant aspect of active databases is their ability to move reactive logic into the data management layer. Instead of having an application repeatedly check whether recent data satisfies some condition, the database can evaluate the condition as data arrives and invoke an action immediately. This is especially attractive for resource-constrained devices, where reducing polling, simplifying application code, and avoiding unnecessary communication are important design goals.

The active rule model considered in this paper is intentionally lightweight. It focuses on insert-triggered rules over recent data, supports conditions expressed through aggregates and predicates, and executes user-defined callbacks as actions. While narrower than the feature set of full active DBMSs, this model captures many common monitoring and control tasks relevant to embedded and IoT workloads.

2.2 Existing Active and Embedded Systems

Active functionality is widely supported in traditional database and stream processing systems, but these systems target deployment environments very different from those considered in this paper. Full database systems such as PostgreSQL and SQLite (Gaffney et al., 2022) support triggers that allow rules to be executed in response to database events. These systems provide expressive rule definitions and mature execution environments, but they assume substantially greater memory, storage, and software support than is available on small embedded devices.

Stream and time series processing systems also support reactive behavior. InfluxDB (InfluxData,

2026a) with Kapacitor (InfluxData, 2026b), for example, allows conditions to be evaluated over incoming data streams and actions to be triggered when thresholds or other criteria are met. This model is effective for server-side monitoring and alerting, but it follows a client-server architecture in which data must first be transmitted to the server before rules are evaluated. For embedded and edge workloads (Kong et al., 2022), this introduces communication overhead and delays action until the data has left the device.

Prior embedded database research has focused primarily on efficient local storage, indexing, and query processing for resource-constrained hardware. These systems demonstrate (Tsiftes and Dunkels, 2011; Douglas and Lawrence, 2014) that useful database functionality can be brought to small devices, but they generally emphasize passive data management rather than integrated active processing. As a result, developers must still implement reactive behavior outside the database by polling for changes or issuing user-level queries after inserts.

The gap addressed in this paper is therefore not the general idea of active rules, which is already well established, but their support in a lightweight embedded database environment. The goal is to provide practical event-driven query processing directly on the device, where rules can be evaluated close to the data source with low overhead and without relying on a remote server or full-scale DBMS.

2.3 EmbedDB Overview

The active rule mechanism in this paper is built on a lightweight embedded database, EmbedDB (Schoenit et al., 2024a; Schoenit et al., 2024b), previously designed for resource-constrained devices. The underlying system supports time series and relational data stored as key-value records, operates with a small memory footprint, and is intended for environments without operating system support. Its main features include efficient inserts, support for key and data-value queries, and compatibility with storage media such as SD cards and raw flash.

The storage architecture uses pre-allocated memory for page buffers, indexes, and system state. Records are written sequentially to storage, which improves insert performance and is well suited to flash-based devices. Optional indexing structures support efficient lookup by key and filtering by data value, allowing the system to avoid scanning unnecessary pages during query execution.

In addition to basic retrieval operations, the system provides a query framework that supports filtering, aggregation, and other relational-style operations over stored records. This query capability is important for the present work because active rules are implemented by evaluating conditions over recent data and then triggering callback actions when those conditions are satisfied.

For this paper, the underlying database serves as the execution substrate for active rule processing. The new contribution is not the storage engine itself, but the addition of active database functionality that allows reactive queries and actions to be evaluated automatically as data is inserted.

2.4 InfluxDB with Kapacitor

InfluxDB (InfluxData, 2026a) is a widely used time series database designed for monitoring, analytics, and event-driven processing in server and cloud environments. Kapacitor (InfluxData, 2026b) is its associated alerting and stream-processing engine, allowing conditions to be evaluated over incoming data and actions to be triggered when those conditions are met. Together, they provide an example of an active time series system that supports rule-based monitoring and notification.

Kapacitor supports ECA-style behavior by allowing active queries to be triggered either on a schedule or after a specified number of inserts, evaluating conditions using queries, calculations, and operators within TICKscript, and executing actions by generating alerts that can be sent to logs, email, external handlers, or third-party services. This makes it a useful comparison point because it supports reactive processing over time series data, but does so in a client-server environment rather than directly on the device.

For the purposes of this paper, the key distinction is architectural. InfluxDB and Kapacitor assume that data is first transmitted to a server, where rules are evaluated and actions are then forwarded to an external process or service. This model is effective for centralized monitoring, but it introduces communication and coordination overhead that is fundamentally different from local rule execution on a resource-constrained device. As a result, InfluxDB with Kapacitor serves as a useful baseline for comparison with embedded active processing, even though it targets a very different deployment setting.

3 Active Rules for Embedded Databases

3.1 Design Goals

The design of the active rule mechanism is guided by four goals. First, it should preserve the low overhead and small memory footprint required for resource-constrained devices. Second, it should support common edge workloads in which recent data must be monitored continuously for threshold conditions or other simple patterns. Third, it should reuse the existing query framework so that active processing can be added without introducing a separate execution engine. Fourth, it should provide a compact programming interface that reduces the need for repeated user-level queries and polling logic in application code.

These goals reflect the intended deployment setting. Many embedded and IoT applications are append oriented, react to recent measurements, and require immediate local actions such as alerts or actuator control. In this setting, a lightweight rule model integrated with insertion is more appropriate than a full active DBMS with richer but more expensive features. The aim is therefore not to replicate the functionality of server-scale active databases, but to provide a practical form of active query processing that fits the constraints of small devices.

3.2 Rule Model

The main contribution of this paper is the addition of active rule support for event-driven query processing on resource-constrained devices. Rules are evaluated after each insertion and are defined over a target column, an optional value filter, a sliding window of recent records, and a comparison predicate. If the condition evaluates to true, a user-defined callback is executed. The rule-building interface has the following structure:

- if: apply an aggregate function
- where: optionally filter rows by a condition
- ofLast: use the last n records
- is: compare the result to a condition
- then: execute an action

Rules are evaluated whenever new data is inserted. The implementation provides predefined aggregate functions, including minimum, maximum, and average, and also supports user-defined custom aggregates. Multiple active rules can be registered and are executed in an application-defined order. Listing 1 shows an example rule that computes the average over the last *N* records and invokes a callback if the result is below a threshold.

Listing 1: Generalized active rule definition.

```
activeRule *rule = createActiveRule(schema,
    context);
rule->IF(rule, columnNumber, GET_AVG)
    ->where(rule, minData, maxData)
    ->ofLast(rule, numLastEntries)
    ->is(rule, LessThan, threshold)
    ->then(rule, callbackFunction);
```

Active databases are generally expected to provide a database model, a way to define and manage ECA rules, mechanisms for detecting events and executing actions, and support for handling multiple rules triggered by the same event.

The proposed design supports these properties in a lightweight form. It builds on the underlying database layer and allows rules to be defined through the active rule interface. Rule designers may use the built-in aggregate functions or have the flexibility to create their own window aggregation functions written in C (`if` clause). It uses the database layer to scan the window of records (`ofLast` clause) and apply filtering conditions (`where` clause) using the relational selection operator. The condition compares the aggregate value defined in the (`if` clause) using a comparator to a threshold value using the `is` clause. If the condition is true, the `then` clause specifies the C callback function to use. The event model supports `INSERT` as the triggering event, while conditions are expressed through the query framework and actions are executed through user callbacks. User callback functions are C functions allowing maximum flexibility on the actions performed. Rules are stored in the database state, can be enabled or disabled dynamically, and are evaluated whenever an insert occurs.

3.3 Supported Query Patterns

The current API is designed primarily for sliding-window analytics and threshold-based actions. It supports common patterns such as computing averages, minima, or maxima over recent records, applying aggregates to filtered subsets of data, defining custom aggregates through the query framework, and coordinating multiple dependent rules through shared callback state. These patterns cover many common edge analytics tasks (Kong et al., 2022), including threshold alerts, anomaly detection, and local actuator control. Shared callback context also allows one rule to pass information to another, enabling more sophisticated conditions and actions than can be expressed by a single rule alone.

4 Experimental Evaluation

4.1 Evaluation Goals and Setup

The evaluation addresses three questions:

1. What is the runtime overhead of active rule execution on embedded hardware?
2. How does the active rule interface compare to implementing equivalent functionality through user-level queries?
3. How does the proposed approach compare to a commercial active time series system, InfluxDB with Kapacitor?

The primary experiment is conducted on an Arduino Due (32-bit, 84 MHz, 96 KB RAM, SD card storage). To model a realistic edge monitoring workload, we use a frost-detection query motivated by vineyard monitoring. In this scenario, a node stores temperature readings and should react when recent measurements indicate sustained freezing conditions. The active rule computes the average temperature over the last n records and invokes a callback when the average falls below 0°C. A sliding-window average is used because it smooths transient noise and better reflects persistent freezing conditions than a single reading. This query follows the rule structure shown in Listing 1, using GET_AVG with a threshold of 0°C. The rule is:

```
IF AVG(temp) OF LAST n RECORDS IS < 0
            THEN callback
```

Temperature values are randomly generated in the range $[-5°\text{C}, 5°\text{C}]$. This causes the sliding-window average to move above and below the freezing threshold during execution, ensuring that the rule is exercised repeatedly during the benchmark rather than never triggering or triggering on every insert. Across the experiments, the callback is invoked about 50% of the time, confirming that the benchmark exercises both the condition evaluation and the action path.

The callback used in the timing experiments performs only a minimal in-memory operation so that the measured time reflects rule evaluation rather than output, communication, or device I/O overhead.

The benchmark begins by inserting 1000 warm-up records. It then inserts another 1000 records while measuring execution time under three configurations: with an active rule enabled, with no active rule enabled, and with the equivalent user-level query issued manually after each insert. For the active-rule and user-level query cases, the sliding-window size is varied over $n \in \{1, 10, 100\}$ to evaluate how performance changes with query window size.

All experiments are repeated three times using different random seeds, and average values are reported. Three runs are sufficient because the variation between runs is very small, with a coefficient of variation below 0.29% in all cases. Using 1000 measured inserts is also sufficient because the per-insert cost is determined primarily by the active query being evaluated, especially the window size, rather than by the total number of records processed during the benchmark. This setup therefore allows the evaluation to measure absolute active-rule overhead on embedded hardware while also characterizing how the cost changes with query complexity.

4.2 Active Rule Overhead on Embedded Hardware

Table 1 summarizes the embedded-device results on the Arduino Due. Overall, the results show that active rule execution is practical on embedded hardware for recent-window monitoring tasks. The baseline system sustains high insert throughput, and enabling the frost-detection rule introduces modest overhead for small windows while still supporting real-time local processing.

As expected, the cost increases with window size because more recent records must be processed on each insert. This makes small windows a good fit for lightweight edge analytics, while larger windows impose substantially higher cost. In particular, the 10-record frost-detection rule represents a realistic workload and still sustains about 1900 inserts per second while evaluating the rule on every insert.

The comparison between active rules and user-level queries helps explain where this cost comes from. Their performance is nearly identical across all window sizes, which shows that the measured overhead is dominated by evaluating the underlying query rather than by the active rule mechanism itself. Active rules are slightly faster in these experiments, likely because the rule is registered once with the system instead of constructing and issuing a user-level query after every insert.

The main advantage of the active rule interface is therefore not lower query cost, but a cleaner programming model for event-driven processing. Common sliding-window conditions and callback actions can be registered once and evaluated automatically, avoiding repeated polling or user-level query orchestration in application code. In addition, registered rules may enable future optimizations that are difficult to apply when equivalent queries are issued manually after each insert.

Table 1: Embedded active rule and user-level query performance on the Arduino Due.

Configuration	Throughput (inserts/s)	Time for 1000 Inserts (s)	Overhead per Insert
No active rule	2951	0.339	–
Window = 1 active rule	2240	0.446	107 µs
Window = 1 user query	2221	0.450	111 µs
Window = 10 active rule	1909	0.524	185 µs
Window = 10 user query	1895	0.528	189 µs
Window = 100 active rule	444.4	2.250	1.911 ms
Window = 100 user query	444.2	2.251	1.912 ms

4.3 Comparison with InfluxDB and Kapacitor

To compare the proposed approach with a commercial active time series platform, a corresponding experiment was conducted using InfluxDB and Kapacitor. Since InfluxDB does not execute on small embedded devices such as the Arduino Due, this comparison was performed on a Windows workstation (Intel i7 2.4 GHz, 32 GB RAM). The experiment follows the same structure as the embedded evaluation, with 1000 initial records inserted and another 10000 inserts used for timing when the active query is enabled.

Table 2 shows a large performance gap between local active rule execution and the InfluxDB/Kapacitor client-server architecture. For a 10-record window, the local implementation sustained about 722,797 inserts per second, compared to 527 inserts per second for InfluxDB/Kapacitor, with average processing times of 0.00139 ms and 1.897 ms per insert, respectively. This difference reflects the execution model more than the rule itself.

In the local approach, the rule is evaluated in process as data is inserted, so the callback can be triggered immediately without network transfer or external coordination. In contrast, InfluxDB uses a client-server architecture in which data must be transmitted to the server, processed by Kapacitor, and then forwarded to an external listener to execute the action. The comparison therefore highlights the practical value of active rules on embedded devices: if rule evaluation is lightweight enough to run where the data is generated, the system can react locally instead of transmitting all measurements to a remote server for processing. For edge and IoT workloads, this can reduce latency and lower communication costs, making embedded active rules a useful alternative to traditional client-server stream processing.

5 Discussion and Limitations

This benchmark represents a common monitoring workload, but it does not cover every possible form of active rule processing. Different aggregates, callback actions, memory budgets, and rule combinations may lead to different absolute overheads. The goal of the evaluation is therefore not to exhaustively measure all rule types, but to demonstrate that active rules can be executed efficiently on embedded hardware and to identify the main factors that influence their cost.

The results provide strong evidence that on embedded hardware, active rules incur modest overhead for small sliding windows and perform nearly identically to equivalent user-level queries, showing that the dominant cost comes from query execution rather than from the rule interface itself. The comparison with InfluxDB and Kapacitor further highlights the benefit of local rule execution. The large performance gap is not simply a matter of implementation efficiency, but reflects the difference between in-process execution at the data source and a client-server architecture that requires external communication.

The design intentionally favors simplicity and efficiency over the broader feature set of traditional active DBMSs. The current system supports only `INSERT`-triggered rules and does not yet include `UPDATE` or `DELETE` events. When multiple rules are present, they are executed sequentially in application-defined order rather than a sophisticated scheduling mechanism.

For the target environment, these restrictions are a reasonable design choice. Many embedded sensing applications are append-oriented and primarily require reactions to newly arriving measurements. In this setting, a narrow but efficient event model is often more useful than a richer active database framework with higher overhead. The results suggest that active rules are especially well suited to lightweight recent-window analytics, threshold detection, and local actuation tasks on resource-constrained devices.

Table 2: PC performance comparison for a 10-record window.

Configuration	Throughput (inserts/s)	Time for 10,000 Inserts (s)	Average Time per Insert (ms)
Active rule	722797	0.0139	0.00139
InfluxDB/Kapacitor	527	18.97	1.897

6 Conclusion

This paper presented a lightweight active rule mechanism for event-driven query processing on resource-constrained devices. The main contribution is the design and implementation of ECA-style active rules for an embedded database environment, including a compact rule interface, sliding-window aggregates, and callback actions. By reusing the existing query framework, the design adds active database functionality while remaining suitable for embedded systems.

Experimental results showed that active rules can be executed efficiently on embedded hardware. On an Arduino Due, a representative frost-detection query with a 10-record window sustained about 1909 inserts per second with an average overhead of 185 µs per insert, while smaller windows incurred even lower cost. Comparison with equivalent user-level queries showed nearly identical performance, indicating that the measured overhead is dominated by query execution rather than by the active rule interface itself. On a PC, the same 10-record rule required about 0.00139 ms per insert, compared to 1.897 ms per insert for InfluxDB with Kapacitor, illustrating the difference between local rule execution and a client-server stream-processing architecture.

These results demonstrate that active database functionality is practical even on highly constrained devices. By evaluating rules directly where data is generated, active rules reduce dependence on external servers, avoid network and cross-process overheads, and enable faster local response. This makes them well suited to edge and IoT applications requiring low-latency monitoring, alerting, and autonomous decision making.

Future work includes optimizing registered active rules, since their structure is known in advance and may permit more efficient execution than equivalent user-level queries. Additional directions include support for richer event models, such as composite or temporal events, more expressive rule conditions, and improved coordination among multiple rules. It would also be valuable to integrate active rules with SQL pre-compilation so that higher-level rule specifications can be translated offline into efficient embedded code for deployment.

EmbedDB with active rule support is available as open source for researchers and developers at https://github.com/ubco-db/EmbedDB.

REFERENCES

Dittrich, K. R., Gatziu, S., and Geppert, A. (1995). The Active Database Management System Manifesto: A Rulebase of ADBMS Features. In *Rules in Database Systems*, volume 985 of *Lecture Notes in Computer Science*, pages 3–20. Springer.

Douglas, G. and Lawrence, R. (2014). LittleD: a SQL database for sensor nodes and embedded applications. In *Symposium on Applied Computing*, pages 827–832.

Gaffney, K. P., Prammer, M., Brasfield, L. C., Hipp, D. R., Kennedy, D. R., and Patel, J. M. (2022). SQLite: Past, Present, and Future. *VLDB*, 15(12):3535–3547.

InfluxData (2026a). Influxdb documentation. https://docs.influxdata.com/influxdb/. Accessed: 2026-03-15.

InfluxData (2026b). Kapacitor documentation. https://docs.influxdata.com/kapacitor/. Accessed: 2026-03-15.

Kong, L., Tan, J., Huang, J., Chen, G., Wang, S., Jin, X., Zeng, P., Khan, M., and Das, S. K. (2022). Edge-computing-driven Internet of Things: A Survey. *ACM Comput. Surv.*, 55(8).

Schoenit, J., Akins, S., and Lawrence, R. (2024a). EmbedDB: A High-Performance Database for Resource-Constrained Embedded Systems Too Small for SQLite. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, SAC '24, page 345–346. ACM.

Schoenit, J., Akins, S., and Lawrence, R. (2024b). EmbedDB: A High-Performance Time Series Database for Embedded Systems. In *Proceedings of the 26th International Conference on Enterprise Information Systems - Volume 1: ICEIS*, pages 240–249.

Tsiftes, N. and Dunkels, A. (2011). A Database in Every Sensor. SenSys '11, pages 316–332, New York, NY, USA. ACM.